Technical Infrastructure for a Licensed AI Data Marketplace

Pay–As–You–Go Architecture, Pipeline, Security, Compliance, and Ops

Internal Engineering Specification

October 16, 2025

Abstract

This document defines a production–grade, pay–as–you–go infrastructure for a data marketplace that (i) ingests creator datasets via a web app, (ii) runs Python algorithms for quality, structure, information–theoretic, and safety checks, (iii) stores and indexes assets by tags/quality/rights, and (iv) enables compliant buying/selling with split payouts. It specifies cloud services, security controls, NSFW/PII safeguards, payments, and operational practices. The design prioritizes EU hosting, least privilege, reproducibility, and auditability.

Contents

1 Goals and Non-Goals

Goals: (1) Smooth creator upload with verifiable provenance; (2) Scalable Python evaluation pipeline; (3) Objective quality/tags with a dataset "Quality Card"; (4) Secure distribution, short–lived access links; (5) Marketplace checkout with split payouts; (6) Built–in NSFW/PII/fairness safeguards; (7) Audit logs for legal compliance; (8) Pay–as–you–go cloud costs.

Non-goals (MVP): Real-time collaborative labeling; on–prem agent deployments; heavy GPU training (beyond light probes).

2 Platform Choice (Pay–As–You–Go)

We anchor the platform on **Google Cloud** (GCP) because of first–class *serverless containers* and per-use billing:

- **Frontend hosting:** Vercel (pay–as–you–go) or Cloud Run (static export/CDN via Cloudflare).
- **API/Backend compute:** Cloud Run (containerized Python/Node; billed per vCPU/GB-second).
- **Batch & pipelines:** Cloud Run Jobs (serverless), optional Vertex AI Workbench only if GPU probes are later needed.
- **Messaging:** Pub/Sub (pay per message).
- **Storage:** Cloud Storage buckets (raw/quarantine/curated), lifecycle rules.
- **Database:** Cloud SQL for Postgres (pay for hours/storage); `pgvector` for embeddings.
- **Cache/Queues (optional):** Memorystore (Redis) for rate limiting/job dedupe.
- **Search/Semantic:** Postgres + `pgvector` initially; scale to Elastic Cloud later if needed.
- **Secrets/KMS:** Secret Manager, Cloud KMS (customer managed keys).
- **Identity/Access:** Google IAM, workload identity federation.
- **CDN & WAF:** Cloudflare CDN (pay–as–you–go) + Cloud Armor (WAF/rate limits) in front of Cloud Run.

All components are available in EU regions (e.g., `europe-west1`) for GDPR–friendly hosting.

3 High-Level Architecture

Key Services

Layer	Service	Purpose
Web & API	Next.js frontend on Vercel or Cloud Run	Creator/buyer UI, SSR SEO
	FastAPI/NestJS on Cloud Run	Auth, REST/GraphQL, signed URLs
Data ingest	Cloud Storage (Signed URLs)	Browser uploads directly to bucket
Async jobs	Pub/Sub + Cloud Run Jobs	Inference, scans, indexing
Data store	Cloud SQL (Postgres + pgvector)	Catalog, users, orders, tags, metrics
Object store	Cloud Storage (raw/quarantine/curated)	Versioned assets, manifests, cards
Search	Postgres full-text / pgvector	Facets, semantic search
Payments	Stripe Connect (primary); Revolut (alt)	Onboarding, split payouts, refunds
Security	Cloud KMS, Secret Manager, Cloud Armor	Keys, secrets, WAF
Observability	Cloud Logging/Monitoring, Sentry	Traces, alerts, error tracking

Data Flow (Narrative)

F1: Upload Init: Authenticated creator requests an upload; backend issues *signed URL* to the `raw/` bucket + an ingest manifest (JSON).

F2: Upload Complete: GCS *object finalize* event emits Pub/Sub message with `dataset_id`, `version`, `paths`.

F3: Quarantine Scan: Cloud Run Job *Ingestor* validates file types, checksums, virus scan, basic schema; moves to `quarantine/`.

F4: Evaluation Pipeline: Cloud Run Job *Evaluator* (Python) loads sample, computes quality/structure/info theory/safety signals, *then* (if needed) runs deep pass; artifacts go to `curated/` with a signed JSON *Quality Card*.

F5: Catalog Indexing: Service writes dataset metadata, metrics, tags, rights, and embedding centroids into Postgres (and `pgvector`); searchable facets are updated.

F6: Listing: If policy passes, dataset becomes *publicly listed*. Otherwise it remains private or requires human review (NSFW/PII flags).

F7: Purchase: Buyer checks out via Stripe; upon successful capture/escrow, backend issues *short-lived signed URLs* for the purchased version and records an immutable license receipt.

4 Detailed Components

4.1 Frontend (Next.js, TypeScript)

- Pages: Creator Dashboard (uploads, versions, payouts), Dataset Editor (metadata, license presets), Listing Page (Data Card + Quality Score), Catalog Search (facets & semantic), Checkout, Buyer Downloads.
- Auth: OAuth2 (Google, GitHub, Microsoft) via NextAuth; organization/team support.
- Upload UX: Chunked direct uploads to signed GCS URL; resumable with content hashes; client–side MIME sniffing.
- Accessibility & SEO: SSR, sitemap, OG tags; dark/light theme; WCAG AA.

4.2 Backend API (FastAPI or NestJS on Cloud Run)

- Endpoints: `/upload/signed_url`, `/datasets`, `/quality_cards`, `/orders`, `/licenses`, `/payouts`, `/webhooks/stripe`.
- Upload orchestration: creates ingest manifests, dataset/version rows, and writes an *expected file list* for integrity checks.
- Policy engine: enforces license/menu presets, territory scopes, "train-only/eval-only", redistribution toggles.
- Access tokens: buyer entitlements map to object paths; signed URL TTLs (e.g., 15 min).

4.3 Async Processing (Pub/Sub + Cloud Run Jobs)

Jobs

- **Ingestor:** filetype/AV scan (`clamav`), schema sniff, checksum, EXIF/C2PA read, staging to `quarantine/`.
- **Evaluator:** runs the two-pass Python pipeline; saves metrics, tags, and a JSON Quality Card; emits alerts for policy flags.
- **Indexer:** computes embeddings (small models), dedupe signatures (LSH/perceptual), then updates Postgres/`pgvector`.

4.4 Datastores

Cloud Storage (GCS)

- Buckets: `raw/`, `quarantine/`, `curated/`, `cards/`, `samples/`.
- Versioning: enabled; lifecycle moves cold versions to Nearline/Archive; optional cross–region replication.
- Object keys: `$dataset_id/$version/$partition/$sha256.ext`.

Postgres (Cloud SQL) — Core Tables (Sketch)

```
-- datasets, versions, assets, tags, metrics, orders, entitlements
CREATE TABLE datasets (
  id UUID PRIMARY KEY, owner_id UUID, name TEXT, modality TEXT,
  visibility TEXT CHECK (visibility IN ('private','listed','delisted'))
     ,
  license_preset TEXT, territory TEXT[], created_at TIMESTAMPTZ,
     updated_at TIMESTAMPTZ
);
CREATE TABLE dataset_versions (
  id UUID PRIMARY KEY, dataset_id UUID REFERENCES datasets(id),
  v INT, status TEXT, quality_overall NUMERIC, grade TEXT,
  card_uri TEXT, created_at TIMESTAMPTZ
);
CREATE TABLE assets (
  id UUID PRIMARY KEY, dataset_version_id UUID REFERENCES
     dataset_versions(id),
  path TEXT, bytes BIGINT, sha256 TEXT, mime TEXT, partition TEXT
);
CREATE TABLE tags (dataset_version_id UUID, tag TEXT, score NUMERIC);
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE embeddings (
  dataset_version_id UUID REFERENCES dataset_versions(id),
  name TEXT, vec vector(384),  -- example dim
  CONSTRAINT embeddings_pk PRIMARY KEY (dataset_version_id, name)
);
CREATE TABLE orders (
```

```
  id UUID PRIMARY KEY, buyer_id UUID, total_cents BIGINT, currency TEXT
     ,
  stripe_payment_intent TEXT, status TEXT, created_at TIMESTAMPTZ
);
CREATE TABLE entitlements (
  order_id UUID REFERENCES orders(id), dataset_version_id UUID
     REFERENCES dataset_versions(id),
  rights TEXT, expires_at TIMESTAMPTZ
);
```

5 Python Evaluation Pipeline (Serverless)

5.1 Tooling and Packages

Core: `numpy`, `scipy`, `pandas`, `scikit-learn`, `statsmodels`, `networkx`.
Quality/Expectations: `great_expectations`, `evidently`.
Info–theory/Similarity: `pyitlib` (or custom KSG MI), `datasketch` (MinHash), `faiss/hnswlib`.
Text: `transformers`, small LMs for embeddings/perplexity; `presidio` (PII); `cleanlab` (label noise).
Images: `opencv-python`, `imagehash` (a/p/pHash), CLIP (ViT-B/32) via `transformers`.
Audio: `librosa`, `jiwer` (WER), `pyannote.audio` (optional diarization).
Security/Safety: regex/NER PII, profanity lists, toxicity NSFW classifiers (e.g., open-source NudeNet/LAION NSFW).
Packaging: Docker images, Poetry/pip-tools; cache models in Artifact Registry.

5.2 Two-Pass Flow (Recap)

Pass 1 (fast): schema & checksums → sample embeddings → missingness/dup/coverage/RAES → leakage screens → info–theory (entropy, MI baseline, compressibility) → safety pre-scan (PII, profanity, NSFW quick).
Pass 2 (deep): full dedupe (ANN/graphs), shift tests (MMD/energy/C2ST), annotation metrics, modality probes (CLIP/ΔPPL/WER), whiteness tests on residuals, calibrated risk.

5.3 Storage by Tags and Quality

After evaluation, the job:

a) writes the JSON *Quality Card* to `cards/`,
b) moves approved assets to `curated/` with metadata (labels, partitions),
c) updates Postgres with tags {`classification`, `RAG`, `ASR`, ...} and normalized scores,
d) sets `dataset_versions.status` to `listed` or `needs_review`.

5.4 Pseudocode (Job Skeleton)

```
def evaluate_and_index(dataset_id: str, version: int, gcs_paths: list[
   str]):
    manifest = load_ingest_manifest(dataset_id, version)
    files = fetch_sample_files(gcs_paths)
    # --- Pass 1 ---
    schema = infer_schema(files); checks = run_checksums(files)
    z = embed(files)            # modality-aware small model
    stats = compute_stats(files, z)    # NE, missingness, dup(LSH),
       RAES, cov, MI(leak), etc.
    safety = quick_safety_scan(files)  # PII/NSFW/toxicity quick
```

```
info = info_signals(files, z)      # entropy, entropy rate,
    compressibility, NCD

deep = {}
if should_run_deep(stats, safety, info):
    deep["dedupe"] = full_dedupe_all(gcs_paths)
    deep["shift"]  = shift_tests(gcs_paths, z)
    deep["annq"]   = annotation_quality(files)
    deep["probes"] = modality_probes(files)
    deep["white"]  = whiten_residuals(files)

scores, tags = aggregate(scores_from(stats,safety,info,deep))
card = build_quality_card(dataset_id, version, scores, tags,
    diagnostics=merge(stats,safety,info,deep))
write_card_to_gcs(card)
route_assets(dataset_id, version, decision=scores["grade"], policy=
    safety["policy"])
index_postgres(dataset_id, version, tags, embeddings=centroid(z))
```

6 Marketplace: Buying & Selling Flow

6.1 Creator Onboarding

1) Sign up → KYC/KYB (Stripe Connect Onboarding).
2) Accept Creator Agreement (IP warranties; right to license; takedown policy).
3) Create dataset; choose license preset (e.g., train–only, eval–only, redistribution policy).
4) Upload; pipeline runs; dataset becomes listed when approved.

6.2 Buyer Journey

1) Search/browse; filter by modality, license, score, risk, tags.
2) View Dataset Page: Quality Card, use–case tags, samples, license summary, change log.
3) Checkout via **Stripe**: payment intent, SCA if required, capture.
4) Upon success: create *entitlements*; issue *short-lived signed URLs*; email/receipt includes license terms and dataset hash manifest.

6.3 Payouts & Refunds

- Split payouts via Stripe Connect: platform fee retained; creator share scheduled T+7.
- Dispute window (e.g., 7–14 days) with evidence (hash match, DOA report, mislabeling).

7 Safety, NSFW, and PII Safeguards

7.1 Upload Gates

- File allowlists per modality (MIME/type); maximum size/chunk; virus scan (`clamav`) on `raw/`.
- Mandatory metadata: provenance attestations, intended use, license preset, territory, TDM status.
- For *potentially sensitive* categories (e.g., medical, NSFW for research): require explicit category selection, stronger review, and locked license scopes.

7.2 Automated Scans (Policy Layer)

- **Text:** PII (names, emails, SSNs; `presidio` + regex), profanity/toxicity (small classifiers); domain–appropriate stoplists.
- **Images:** NSFW classifiers (e.g., open NudeNet/LAION models), face detection; EXIF/C2PA provenance checks.
- **Audio:** VAD segmentation, profanity detection, diarization hints; SNR thresholds; speech/non-speech gating.
- **Medical Exceptions:** require a medical–use flag, de–identification attestations, and stricter license + access controls (gated audience).

7.3 Human Review & Enforcement

- Any high–severity flag (NSFW without proper category, PII above threshold, child safety concerns) routes to *manual review queue*.
- Repeat infringer policy; automated delisting; evidence preserved (hashes, sample fingerprints).

8 Security Architecture

8.1 Principles

Least privilege, default deny, end–to–end encryption, short–lived credentials, zero trust between services, reproducible builds.

8.2 Controls

- **Identity:** OAuth for users; IAM service accounts for jobs; Workload Identity for CI/CD.
- **Network:** Private Serverless VPC; restrict Cloud SQL to private IP; Cloud Armor WAF/rate limits; per–tenant API throttles via Redis.
- **Data:** CMEK via Cloud KMS for buckets and SQL; object encryption at rest and TLS 1.2+ in transit; signed URLs with short TTL; hash manifests for integrity.
- **Secrets:** Secret Manager; no secrets in images; rotation policies; SCIM SSO for staff dashboards.
- **AuthZ:** RBAC: owner, collaborator, reviewer, buyer; dataset–scoped ACLs; entitlement checks on every download.
- **Supply chain:** SBOM (Syft), image scanning (Trivy), provenance attestations (SLSA).

8.3 Backups & DR

- Postgres PITR; daily snapshots; tested restores.
- GCS versioning + lifecycle; optional cross–region backups (cold).
- Runbooks for KMS key recovery, credential compromise, and data breach notifications.

9 Observability and Reliability

- **Logs & Metrics:** Cloud Logging + OpenTelemetry traces; per–job metrics (latency, error ratio, cost).
- **Alerting:** SLOs: API p95 <300 ms; ingest start <30 s; evaluation TTR (time–to–result) <30 min for 10 GB datasets. Pager on sustained errors.
- **App QA:** Synthetic checks for uploads, checkout, signed URL access; contract tests for webhooks.

10 Compliance Hooks

- **Auditability:** Immutable logs of uploads, scans, decisions; Quality Cards include metric provenance and code image digest.
- **Privacy:** DSAR endpoint; DPA templates; region pinning (EU buckets); SCC/IDTA templates for cross–border transfers.
- **TDM/License:** per–dataset TDM status; opt-out detection and storage; license receipts attached to orders.
- **Notice-and-Action:** takedown portal (DMCA/DSA), repeat infringer handling; evidence retention.

11 Cost Controls (Pay–As–You–Go)

- Serverless compute (Cloud Run/Jobs) autoscale to zero; per-second billing.
- GCS lifecycle to Nearline/Archive for old versions; signed URL egress only after purchase.
- Embeddings cached; cap deep–pass samples; batch large jobs; per–tenant quotas.
- One database (Cloud SQL) to start; postpone Elastic cluster until search volume justifies.

12 DevOps and CI/CD

- **IaC:** Terraform modules for projects, IAM, Cloud Run, SQL, Pub/Sub, buckets, KMS.
- **CI/CD:** GitHub Actions → Build Docker → Trivy scan → push to Artifact Registry → deploy to Cloud Run (blue/green).
- **Migrations:** `alembic/prisma` migrations; schema versioning.
- **Data versioning:** object versioning + manifest files; optional DVC/lakeFS later.

13 Risk Register (Selected)

Risk	Mitigation	Owner
Abuse uploads (illegal/NSFW)	Multi-model safety scans + manual review + repeat-infringer policy	Trust & Safety
PII spill	PII detectors + thresholds + de-ID tooling + human review	Data Eng
License disputes	Clear creator warranties; takedown portal; escrow window	Legal
Cost spikes	Autoscale to zero; lifecycle storage; quotas	SRE
Data leakage (downloads)	Signed URLs; short TTL; per-download logging; WAF	Platform

14 Roadmap (Infrastructure)

Q1: **Q1:** Core buckets, Cloud Run (API + Jobs), Postgres, Stripe Connect, upload → evaluate → list → buy flow, basic safety scans.

Q2: **Q2:** Semantic search (pgvector), C2PA reading/writing, expanded safety models, Data Plus channel, audit receipts.

Q3: **Q3:** Elastic Cloud (if needed), SOC2-lites, cross-region backups, private deals workflow, enterprise SSO.

15 Appendix A: Default NSFW/PII Policy Thresholds (Illustrative)

Signal	Action	Notes
Text PII score $>T_{\text{PII}}$	Quarantine & de-ID required	`presidio`+regex; human review if borderline
Image NSFW prob >0.8 (no medical flag)	Block listing	Route to review if creator disputes
Audio profanity ratio $>r$	Flag + license restriction	Gated tag (mature/explicit)
Child safety any positive	Immediate block + escalate	Mandatory report policies

16 Appendix B: Example Terraform Module Sketch

```
module "buckets" {
  source      = "./modules/gcs"
  project_id  = var.project_id
  buckets     = ["raw", "quarantine", "curated", "cards", "samples"]
  location    = "EUROPE-WEST1"
  versioning  = true
  kms_key     = google_kms_crypto_key.data_key.id
}

module "cloud_run_api" {
  source        = "./modules/cloud_run"
  image         = "europe-docker.pkg.dev/.../api:latest"
  min_instances = 0
  max_instances = 10
  vpc_connector = google_vpc_access_connector.svpc.name
  env           = { DATABASE_URL = var.db_url, STRIPE_KEY = var.stripe
     }
}
```

17 Appendix C: Quality Card (JSON Schema Sketch)

```
{
  "$schema":"http://json-schema.org/draft-07/schema#",
  "title":"QualityCard",
  "type":"object",
  "properties":{
    "dataset_id":{"type":"string","format":"uuid"},
    "version":{"type":"integer"},
    "scores":{"type":"object"},
    "diagnostics":{"type":"object"},
    "tags":{"type":"array","items":{"type":"object"}},
    "provenance":{"type":"object"},
```

```
    "legal":{"type":"object"},
    "hash_manifest":{"type":"array","items":{"type":"string"}}
  },
  "required":["dataset_id","version","scores","diagnostics","tags"]
}
```

18 Appendix D: Minimal API Endpoints

Endpoint	Method	Purpose
`/upload/signed_url`	POST	Return GCS signed URL + manifest
`/datasets`	POST/GET	Create/list datasets
`/datasets/{id}/versions`	POST/GET	New version, list versions
`/quality_cards/{id}`	GET	Fetch Quality Card
`/search`	POST	Facet + semantic search (pgvector)
`/checkout`	POST	Create Stripe payment intent
`/webhooks/stripe`	POST	Handle payment/payout events
`/downloads/{id}`	POST	Issue short-lived signed URLs

Summary. The above architecture delivers a compliant, secure, and cost–efficient marketplace: creators upload once; Python jobs assess quality and safety; assets are stored and indexed by tags/quality/rights; buyers receive auditable licenses and short–lived access, with built–in safeguards against unsafe or unlawful content.